SYMMETЯIC

powered by ***Blaze***

A High-Speed Reversible Constrained Random ARMv8 Test Generator

built by **DHILAN RAMAPRASAD**

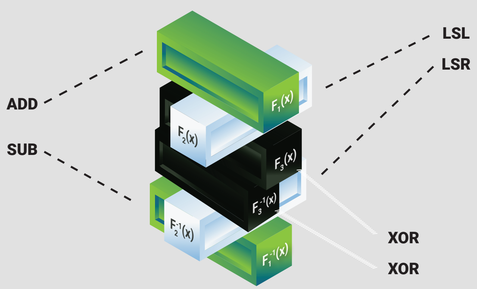
mentor: **TOM HART**
manager: **RAVI SREE**

SYMMETЯIC enables functional validation without accompanying RTL simulation (for state comparison) thereby leveraging rapid test generation alongside runtime verification which is *theoretically* orders of magnitude faster than traditional post-silicon validation methodologies.

First introduced by **ILYA WAGNER** and **VALERIA BERTACCO**, I employ a generation framework in SYMMETЯIC—which they called ***Reversi***—to create a "known-correct" final state.

FRAMEWORK

SYMMETЯIC PAIRS & FUNCTIONAL BLOCKS



Illustrated is an example of a ***single stack*** *operating on one destination* ***"focus" register*** *whose initial state is preset and should match its final state ("known correct") after a series of* ***nested sequences*** *of Arm instructions and their inverses.*

[**Pair:** forward, then reverse (undo or inverse)]

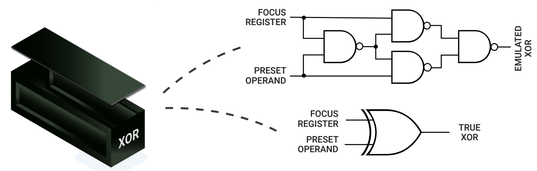
COVERAGE

ADD REG, **ADD** IMM	**SUB** REG, **SUB IMM**	32,64
MUL, UMADDL REG	**UDIV** REG	32,64
LSL REG, LSL IMM	**LSR** REG, **LSR** IMM	32,64
AND, ORR BITMASK, **BFM/BFC**		32,64
AND, ORR, ORN, MVN, EON, EOR REG		32,64

and quickly growing to include alias instructions and complex instructions which will test short/long flow + Estes HW decoder, too.

UNDER THE HOOD

Many instructions in the Arm ISA are **lossy** (e.g., shifts) or **non-invertible** (e.g., AND). In those cases, a functional block is composed of multiple instructions to create a reversible sequence via emulation of another instruction (e.g., emulated exclusive or using AND and NOT) or storage and restoration of lost bits.



Compound functional blocks also afford an opportunity to identify, enumerate, and implement a number of alternative reversible pairs, mixing immediate and register type instructions to increase overall randomization and **verify functional behavior**, ensuring that a pair is not simply correct in that it merely reverses itself.

50+ COMBINATIONS TO EMULATE **XOR**

RUNTIME CHECKER

The value proposition of SYMMETЯIC is that it retains radomization in test generation without requiring simultaneous architectural simulation to ensure proper functionality. Because the initial state of a focus register must match its end state, a simple equality check can be performed periodically after a stride of forward instructions has been unrolled via their reverses.

initial state == **end state**

CONTROL FLOW

In hand-coded Arm assembly, we have confirmed that taken and not-taken branch (conditional) and jump instructions can be deployed with a **modifying** arithmetic instruction or an end-of-test halt in the forward direction with a link to a later emplaced (i.e., not known at deployment of initial branch) inverse instruction containing an opposite style branch as well as a **restorative** arithmetic instruction or an end-of-test halt for incorrect operation. Wagner's *Reversi* relies on modifying/restorative loads; whereas, SYMMETЯIC leverages arithmetic instruction pairs to validate behavior.

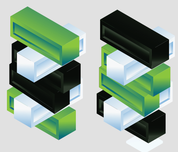
UNEXPLORED

Because other tests in the ***Blaze*** test generation suite offer coverage of memory, creating **Load/Store** operational blocks has been lower priority. However, an expected framework includes pre-filling memory and performing alterations and subsequent restorations such that an XOR-hash (as suggested in *Reversi*) for initial and final state comparison will reveal correct operation.

Floating point instructions present the largest challenge as estimation and tolerances will need to be devised for reversal.

MULTI-STACK

By creating disjoint sets of GPRs, we also initialize multiple focus registers and ***interleave multiple symmetric stacks*** *at a time during generation for further randomization.*



PERFORMANCE

Because SYMMETЯIC performs a self-contained runtime check, ***bottlenecks*** of **on/off-loading tests** to and from silicon as well as runnning "golden model" **architectural simulation** for behavior validation can be avoided in whole while still performing functional tests.

Reference points for average simulation time of a **25,000 instruction** reversible test on core testbench and ccplex:

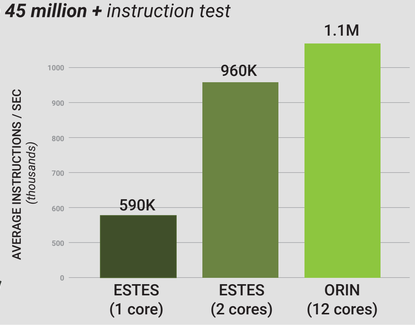
18.6 *minutes* CORE TB | **53.1** *minutes* CCPLEX

too slow

TEST STATISTICS

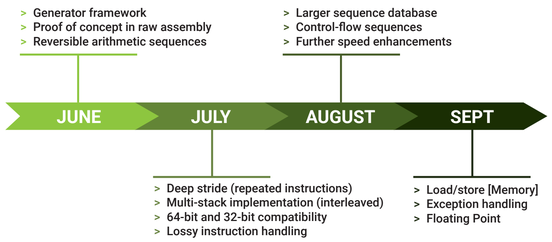
- *Theoretically* ***infinite*** *(memory-constrained) nested forward stride before reversal if composed of non-lossy instructions (64-bit, no shift)*
- *Minimum* ***15-pair stride*** *(all lossy)*
- *Demonstrated ability to generate* ***45 million +*** *instruction test*
- ***3000x faster*** *(on average) than* ***Grind*** *generator—though less targeted at specific scenarios, but randomization and volume of stimulus via high IPS fills the difference.*
- ***14.48kB per 1M*** *instructions*

GENERATION SPEED AT A GLANCE



TIMELINE + UP NEXT

GENERATOR BUILD PROGRESS



USAGE PROGRESS

1. Tests passing on ASIM, DSIM, and during nightly qualifying runs on core and ccplex testbenches.
2. Find bugs on RTL by disabling runtime checkers and only using SYMMETЯIC end-of-test state verification.
3. Executing randomly generated tests on emulation, FPGA, and finally silicon (after tape-out) will offer many of the optimal, rapid use-cases intended for SYMMETЯIC.

CONCLUSION

Through a verified proof-of-concept build and assorted layerings arithmetic instruction sequences, we have shown the **speed** and inherent **efficacy** of a constrained random test generation method founded on SYMMETЯIC (*Reversi*) framework wherein a core's final state is known at the time of test generation. RTL simulation can be circumvented, via the symmetric generator, when finding bugs during functional validation.

REFERENCES

I. Wagner and V. Bertacco, "Reversi: Post-silicon validation system for modern microprocessors," 2008 IEEE International Conference on Computer Design, Lake Tahoe, CA, 2008, pp. 307-314, doi: 10.1109/ICCD.2008.4751878.

ABOUT ME

Dhilan Ramaprasad from **Morristown, Tennessee**
Rising Senior studying Electrical Engineering and CS
Expected Graduation **December 2021**
Hobbies: teaching, video production, auto-repair, politics

Pictured inside my NVIDIA workstation shipment box on Day 1 of my summer virtual internship experience.